

1 实现漫反射康奈尔盒场景下的光子图建立算法

光子映射算法是对路径追踪的一种改进，采用两阶段方法实现光照模拟。

第一阶段，从光源发射 n 个携带能量的光子。光子在场景中传播，遇到镜面或折射表面时沿相应方向继续行进；若击中漫反射表面，则将其能量、位置和入射方向记录于全局光子图中，表示该光子能量被表面吸收。此后，光子可能被完全吸收，或更新能量并沿随机方向继续传播。此阶段持续至所有光子均被漫反射表面吸收，最终生成包含全部光子信息的光子图。

第二阶段与路径追踪类似，从像素发射光线并进行追踪。不同之处在于，光子映射在光线与表面交点处，利用邻近 k 个光子的能量与方向信息估算局部光照。该方法通过光子密度估计光照强度，相比基于蒙特卡洛积分的随机采样更为高效、稳定。

综上，光子映射的第一阶段构建光子分布图，记录光子在漫反射表面的能量沉积；第二阶段则借助该光子图进行光照估算，有效提升渲染质量与效率。

遍历场景的面积光源 (Cornell box 顶灯)，根据 $\text{area} = u \times v$ 和 radiance 计算每个光源的理论功率 $\text{power} = \text{radiance} * \text{area} *$ 。用亮度作为权重，把配置里的 photonCount 按比例分配给各个光源，并记录理论总能量 expectedEmit 。

对每个光源生成对应数量的光子。采样 (rx, ry) 均匀落在光源平面上，使用半球采样 (缺省 Hemi-Sphere) 在光源法线半空间里随机出射方向。每个光子携带固定能量 $\text{fluxPerPhoton} = \text{power} / \text{photonsForLight}$ 。

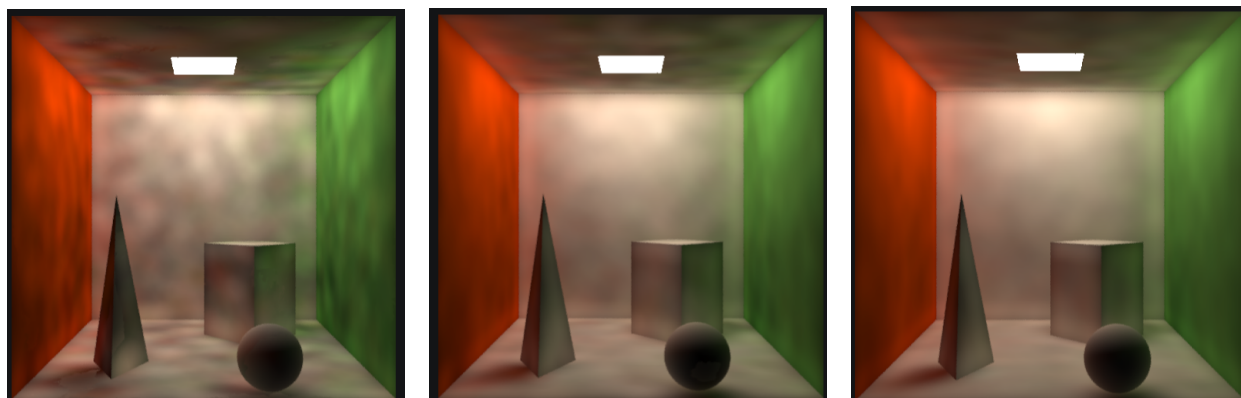
光子沿着场景几何体前进，最多 20 次弹射；如果命中漫反射表面，就把光子位置、入射方向和当前能量存进 photons 列表，用于后续查询；同时根据 albedo 记录吸收能量，并以俄罗斯轮盘决定是否继续随机散射。遇到镜面或介质材质则按 Fresnel 概率决定反射或折射，不在这些表面存储光子，只改变方向继续追踪；若光子射出场景视为逃逸，能量计入 sumEscapedFlux 。

所有存下来的光子通过 PhotonMap::build 用中位数划分构建 KD-tree；每个节点存包围盒、轴、索引，便于之后的球半径查询。

2 不同数量的光子 (10000-100000) 的光子图可视化输出

接着，我们来看渲染的效果，所有的半径都为 50。

我们先来看漫反射场景的康奈尔盒 (`path_tracing_cornel.scn`)，渲染效果如图1所示，对应的光子图如图2所示：

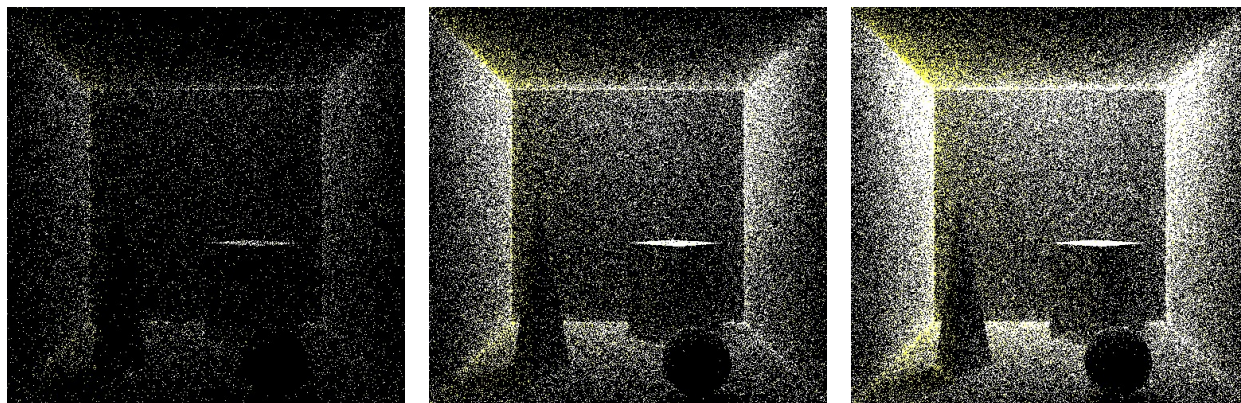


10000 光子

50000 光子

100000 光子

Figure 1: 漫反射场景的康奈尔盒渲染结果



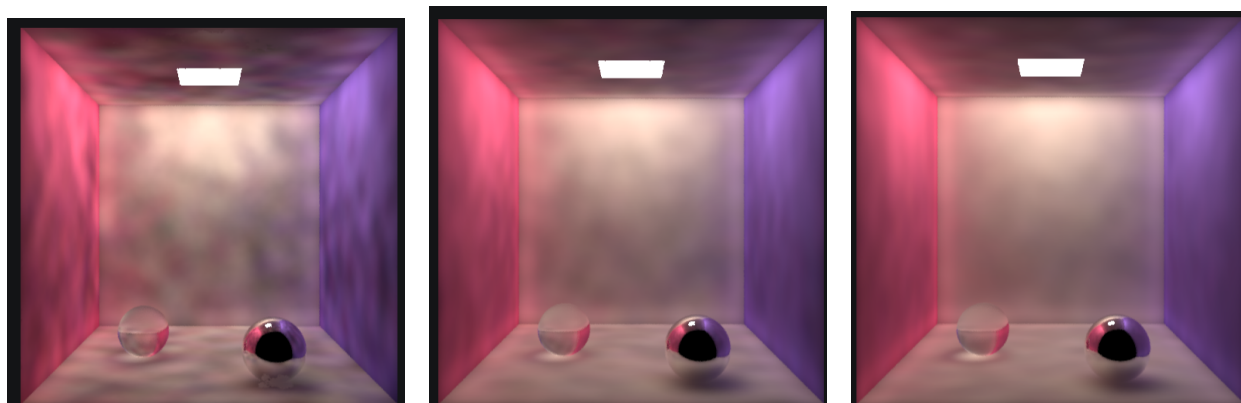
10000 光子

50000 光子

100000 光子

Figure 2: 漫反射场景的光子图

同时我们换一个康奈尔盒场景，场景中有一个玻璃球（左），反射球（右），渲染结果如图3所示，对应的光子图如图5所示：

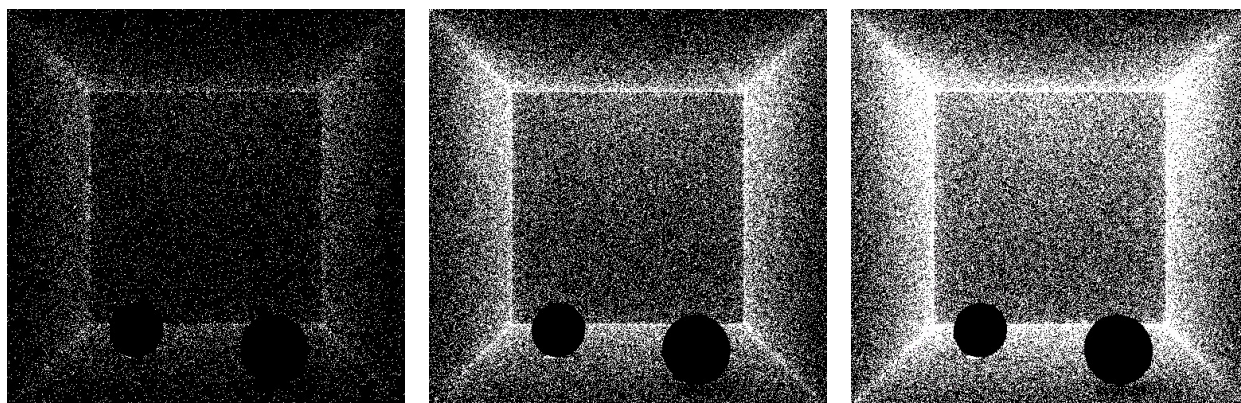


10000 光子

50000 光子

100000 光子

Figure 3: 有反射与折射的康奈尔盒渲染结果



10000 光子

50000 光子

100000 光子

Figure 4: 有反射与折射的康奈尔盒光子图

我们可以看到随着光子数量的增加，降低了密度估计的方差，从而实现了更平滑、更连续的亮度

估算。

3 统计并验证场景在不同数量光子下是总能量守恒的

根据 1 里面的实现，我们不难看出能量应该满足发射的光子能量 = 漫反射表面接受的能量 + 逃逸出场景的光子的能量。

于是我们就可以根据公式 $\text{sumInitial} = \text{sumEmitted} = \text{sumAbsorbed} + \text{sumEscaped}$ ，来计算能量是否守恒。我们利用 $(\text{sumAbsorbed} + \text{sumEscaped})/\text{sumInitial}$ 来计算能量是否守恒（越接近 1 约守恒）

在日志中可以观察到：

```
[ Sun Nov 30 14:41:44 2025 ] [PhotonMap] expected emit power = (2164158.250000, 1744702.750000, 1406062.250000)
[ Sun Nov 30 14:41:44 2025 ] [PhotonMap] emission / Initial = (1.000000, 1.000000, 1.000000)
[ Sun Nov 30 14:41:44 2025 ] [PhotonMap] Global / Initial = (0.995832, 1.004169, 1.004464)
```

10000 光子

```
[ Sun Nov 30 14:48:31 2025 ] [PhotonMap] expected emit power = (2164158.250000, 1744702.750000, 1406062.250000)
[ Sun Nov 30 14:48:31 2025 ] [PhotonMap] emission / Initial = (1.000000, 1.000000, 1.000000)
[ Sun Nov 30 14:48:31 2025 ] [PhotonMap] Global / Initial = (1.001548, 1.001856, 1.000940)
```

50000 光子

```
[ Sun Nov 30 15:06:23 2025 ] [PhotonMap] expected emit power = (2164158.250000, 1744702.750000, 1406062.250000)
[ Sun Nov 30 15:06:23 2025 ] [PhotonMap] emission / Initial = (1.000000, 1.000000, 1.000000)
[ Sun Nov 30 15:06:23 2025 ] [PhotonMap] Global / Initial = (1.002007, 1.000628, 1.000787)
```

100000 光子

Figure 5: 渲染结果日志

我们可以看到，最后的结果比值都接近 1，说明不同数量光子下是总能量守恒的。